



PELO FUTURO DO TRABALHO



Professor



- Professor nas áreas de tecnologia, administração e educação.
- Experiência docente no SENAI, Instituto Federal Catarinense e Uniasselvi.
- Mestrando em Educação (Instituto Federal Catarinense)
 - Especialização em Educação a Distância na Educação Profissional e Tecnológica (IFC)
 - Especialização em Abordagens Ativas e suas metodologias (IFSC)
 - MBA em Gestão de Projetos (Uniasselvi)
 - MBA em Gestão de Tecnologia da Informação (IGTI)
 - MBA em Gestão Estratégica de Pessoas (Estácio)
- Graduação em Sistemas de Informação
- Licenciatura em Matemática
- Pesquisa voltada para Inteligência Artificial e inclusão.
- Atua com pesquisa, educação profissional, tecnologias educacionais e inovação na educação.

Objetivo

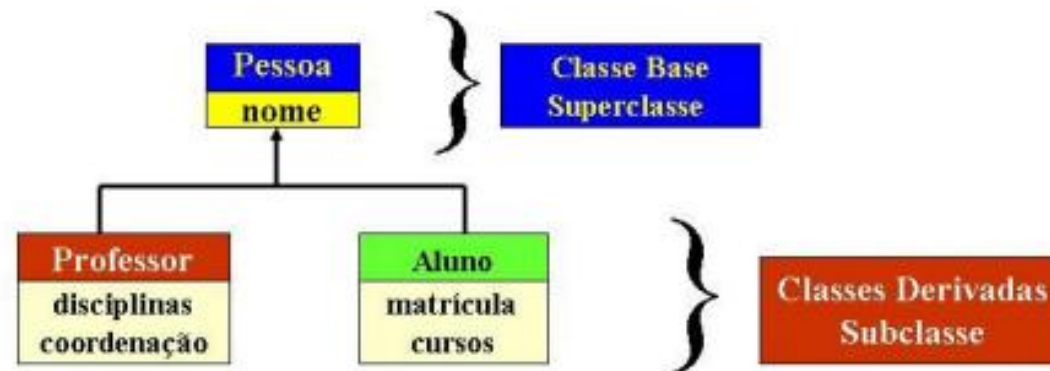
- Conceitos de herança e polimorfismo

Herança

As classes podem ser organizadas de forma a constituírem famílias que compartilham propriedades e operações. A partir de conjuntos de operações comuns a duas ou mais classes, pode-se estruturar uma hierarquia de classes, na qual cada classe define suas próprias operações, que também são herdadas por suas subclasses. Dessa forma, objetos pertencentes a classes inseridas em uma hierarquia podem ser utilizados em contextos onde se espera um objeto de uma classe mais geral, desde que haja relação de herança.

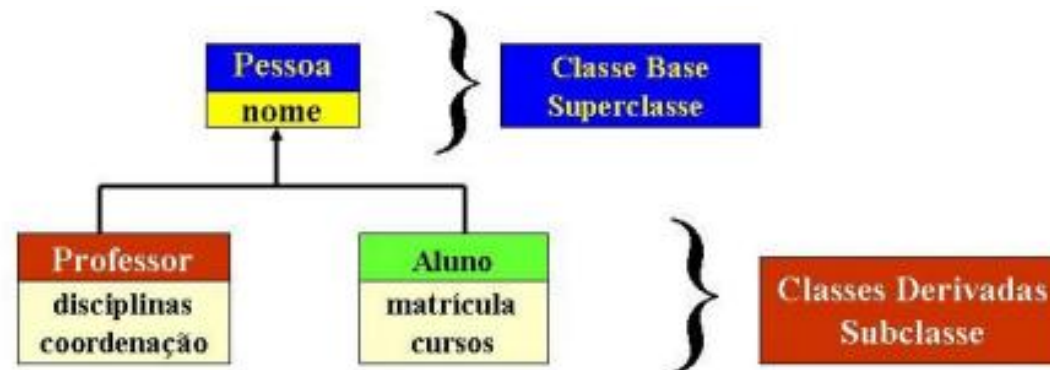
Hierarquia

Ao declarar uma nova classe, é possível reutilizar a definição de atributos e métodos de uma classe previamente existente. Esse mecanismo de reaproveitamento de implementação é denominado herança de classe. A nova classe é chamada de classe derivada ou subclasse, enquanto a classe original é denominada classe base ou superclasse. A subclasse herda todas as características da superclasse e pode estendê-las por meio da adição de novos atributos e métodos.



Hierarquia de classes

Por exemplo, objetos do tipo Pessoa possuem o atributo nome. Já os objetos da classe derivada Professor possuem os campos nome, disciplinas e coordenação, enquanto os objetos do tipo Aluno, também derivados de Pessoa, possuem os atributos nome, matrícula e cursos.



Especialização e Generalização



Especialização e Generalização



Interfaces

Os tipos classificam os dados, fornecendo uma organização para os valores utilizados em um programa. A separação dos dados em tipos, como inteiros, fracionários, cadeias de caracteres e booleanos, permite uma implementação mais eficiente das operações de cada tipo, além de facilitar a detecção de erros decorrentes do uso inadequado de valores.

Todo tipo possui uma interface que deve ser claramente especificada. Em Java, a construção interface é um recurso da linguagem utilizado para definir as assinaturas das operações de tipos abstratos de dados, que serão implementadas posteriormente por meio da declaração de classes.

Mas funciona sem interface?

Sim.

O problema surge quando o sistema cresce. Sem o uso de interfaces, o código tende a ficar mais acoplado, dificultando alterações, reutilização e expansão do sistema. Além disso, a realização de testes também se torna mais complexa, reduzindo a flexibilidade e a manutenção do software.

Implementação de Interface – EXEMPLO

Teórico de Aplicabilidade

```
1 public class Bolo {
2     private double calorias, gorduras;
3     public Bolo(double c, double g) {
4         calorias = c; gorduras = g;
5     }
6     public String identidade() {return "Bolo" ;}
7     public double calorias() {return this.calorias;}
8     private double gorduras() {return this.gorduras;}
9 }
10 public class Torta {
11     private double calorias, gorduras;
12     public Bolo(double c, double g) {
13         calorias = c; gorduras = g;
14     }
15     public String identidade() {return this.calorias;}
16     private double gorduras() {return this.gorduras;}
17 }
```

Código Bagunçado

```
1 public class Usuário1 {
2     private double g1(Bolo y) {return y.calorias();}
3     private double g2(Torta y) {return y.calorias();}
4     public void main() {
5         PrintStream o = System.out;
6         Bolo b = new Bolo(150.0, 19.0);
7         Torta t = new Torta(300.0, 56.0);
8         double z = g1(b) + g2(t);
9         o.println(b.identidade() + ":");
10        o.println(" " + b.calorias() + " kcal");
11        o.println(" " + b.gorduras() + "g de gorduras");
12        o.println(t.identidade() + ":");
13        o.println(" " + t.calorias() + " kcal");
14        o.println(" " + t.gorduras() + "g de gorduras");
15        o.println("Total de Calorias de = " + z);
16    }
17 }
```

Implementação de Interface – EXEMPLO

Teórico de Aplicabilidade



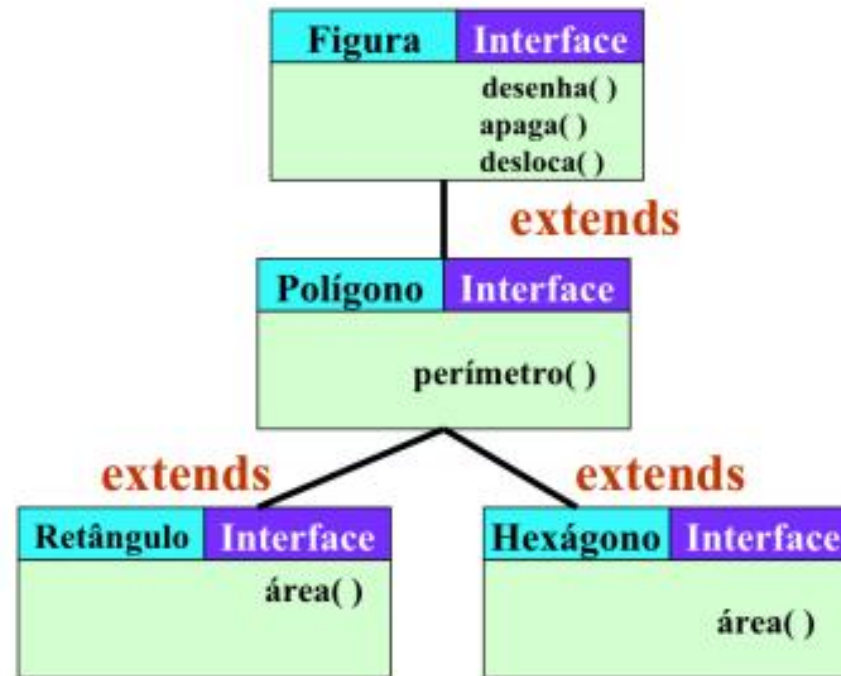
```
1 public interface Comestível {  
2     double calorias();  
3     String identidade();  
4     double gorduras();  
5 }
```

```
1 public class Bolo implements Comestível {  
2     private double calorias, gorduras;  
3     public Bolo(double c, double g) {  
4         calorias = c; gorduras = g;  
5     }  
6     public String identidade() {return "Bolo";}  
7     public double calorias() {return calorias;}  
8     public double gorduras() {return gorduras;}  
9 }
```

Solução

```
1 public class Usuário2 {  
2     private double g (Comestível y) {  
3         return y.calorias();  
4     }  
5     public void main(String [] args) {  
6         PrintStream o = System.out;  
7         Comestível b = new Bolo(150.0, 19.0);  
8         Comestível t = new Torta(300.0, 56.0);  
9         double z = g(b) + g(t);  
10        o.println(b.identidade()+":"+b.calorias+","+b.gorduras);  
11        o.println(t.identidade()+":"+t.calorias+","+t.gorduras);  
12        o.println("Total de Calorias de = " + z);  
13    }  
14 }
```

Hierarquia de Interfaces



Classe Abstrata

As interfaces em Java são um recurso de programação que permite o desenvolvimento de sistemas com alto grau de abstração. Os usuários de uma interface conhecem apenas as assinaturas dos métodos e as constantes por ela definidas, podendo abstrair-se completamente dos detalhes de implementação.

Classe Abstrata

Classes abstratas são identificadas pela palavra-chave `abstract`. Elas definem um conjunto de membros comuns às suas subclasses, além das assinaturas de métodos que deverão ser implementados posteriormente.

As classes abstratas possuem estrutura semelhante à de uma classe comum, porém podem conter um ou mais métodos sem corpo, chamados de métodos abstratos. Dessa forma, representam definições incompletas de classes, servindo como base para especializações por meio da herança.

Classe Abstrata

Classes abstratas podem possuir construtores, porém não é possível criar objetos diretamente a partir delas. Esses construtores são utilizados apenas quando acionados pelas construtoras de suas subclasses.

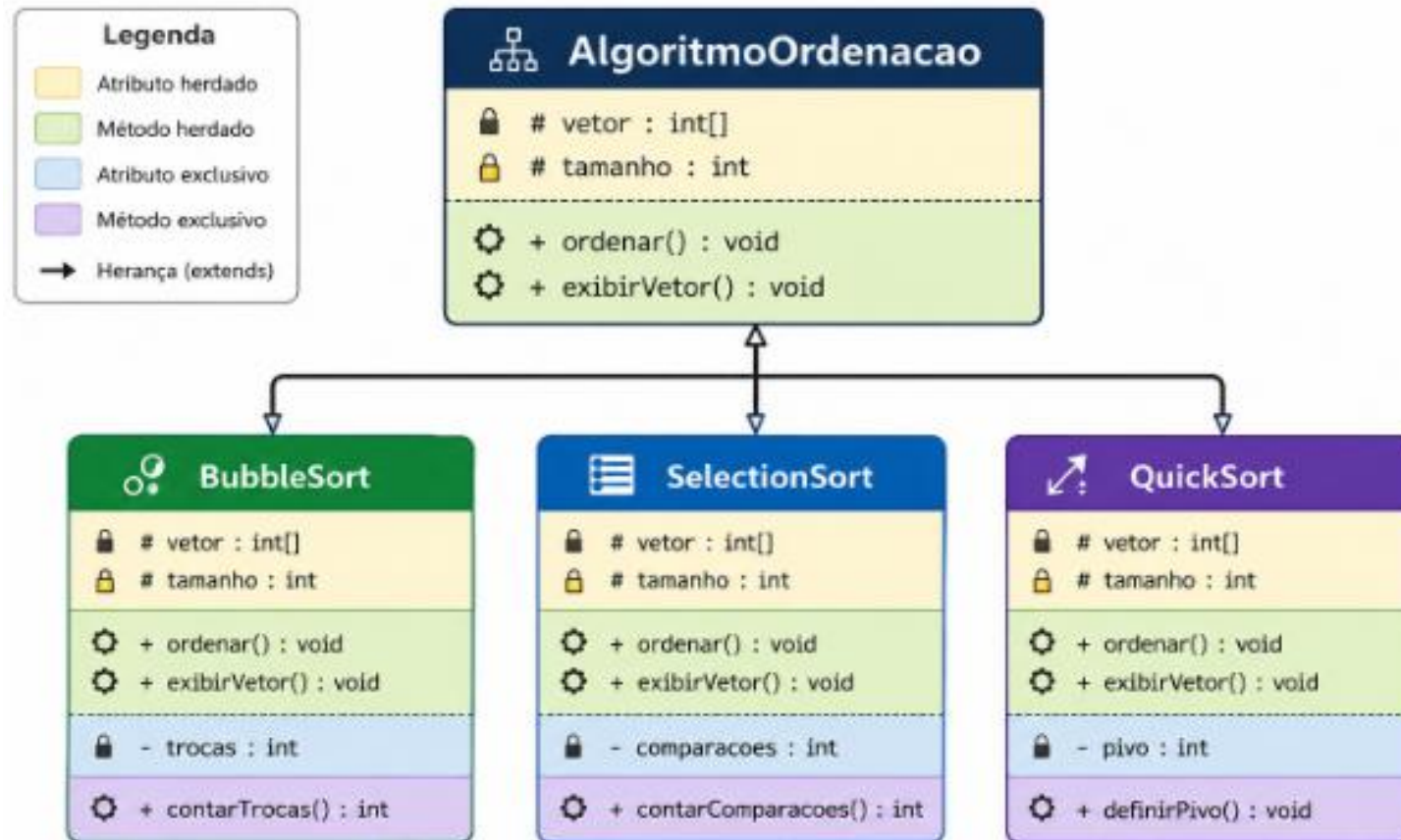
Classe Abstrata

```
1 public class Círculo extends Forma {
2     private double raio;
3     public Círculo(Color c,int x,int y,double raio) {
4         super(c, x, y); this.raio = raio;
5     }
6     public Círculo() { this(Color.red, 0, 0, 1);}
7     public double área () {return 3.1416*raio*raio;}
8     public void desenha() { ... }
9 }
10 public class Retângulo extends Forma {
11     private double largura;
12     private double altura;
13     public Retângulo
14         (Color c, int x, int y, double a, double b) {
15         super(c, x, y); this.largura = a; this.altura = b;
16     }
17     public void área() {return largura*altura;}
18     public void desenha() { ... }
19 }
```

Execução de construtoras

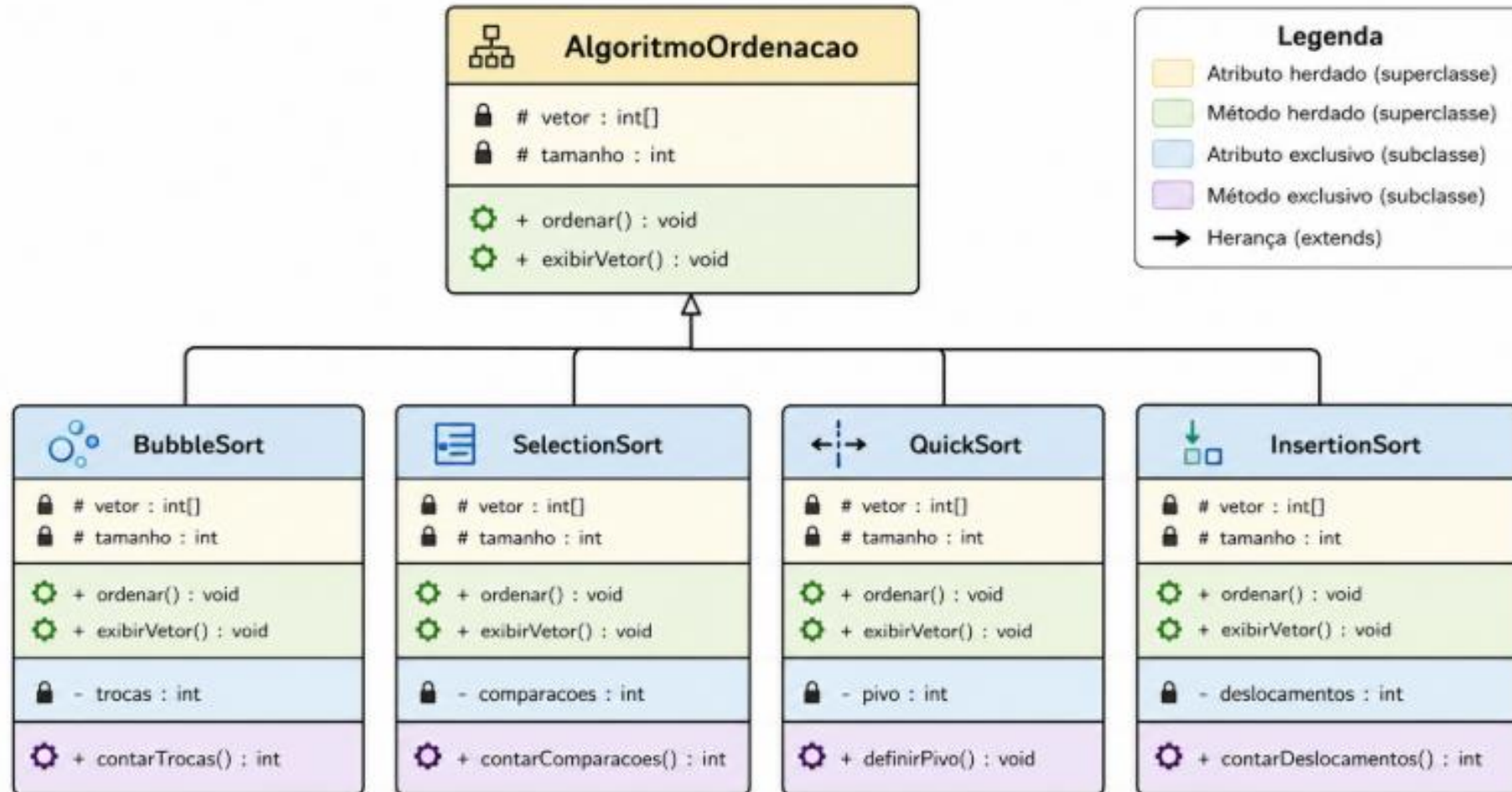
```
1 class A {  
2     public A() {  
3         System.out.println("Passei na A()");  
4     }  
5 }  
6 class B extends A {  
7     public B (int a) {  
8         super();  
9         System.out.println("Passei na B(a)");  
10    }  
11    public B() {  
12        this(0,1);  
13        System.out.println("Passei na B()");  
14    }  
15    public B(int a, int b) {  
16        System.out.println("Passei na B(a,b)");  
17    }  
18 }  
19 public class C {  
20     public static void main (String[] args) {  
21         System.out.println("Comecei na C");  
22         B b1 = new B();  
23         System.out.println("Voltei na C");  
24         B b2 = new B(1,2);  
25         System.out.println("Terminei na C");  
26     }  
27 }
```

Exemplo



Resumo: BubbleSort, SelectionSort e QuickSort herdam os atributos e métodos da classe **AlgoritmoOrdenacao** (vetor, tamanho, ordenar(), exibirVetor()). Cada classe possui ainda atributos e métodos exclusivos.

Atividade



Atividade

Qual classe representa a superclasse do sistema?

Quais atributos foram herdados pelas subclasses?

O que é sobrescrita de métodos?

Por que o método ordenar() possui implementações diferentes?

Qual a vantagem da herança nesse exemplo?

This work is licensed under a Creative Commons license



Attribution-ShareAlike 4.0

<http://creativecommons.org/licenses/by-sa/4.0/>



PELO FUTURO DO TRABALHO

sesisc.org.br     **sc.senai.br**

0800 048 1212

Rodovia Admar Gonzaga, 2765 - Itacorubi - 88034-001 - Florianópolis, SC